HRLO: Hierarchical Refinement Logit Optimization

A Model-Agnostic Inference Acceleration Framework for Large Language Models

Preliminary Technical Proposal • March 2026

Abstract

We propose HRLO (Hierarchical Refinement Logit Optimization), a novel inference acceleration framework for large language models (LLMs) that operates entirely outside the base model's weights. HRLO addresses the vocabulary projection bottleneck — the O(d·V) cost of the lm_head linear transformation and softmax over large vocabularies — by using a hierarchy of lightweight auxiliary layers to progressively filter the candidate token space from the full vocabulary V down to a small subset of size K, upon which the base model M_0 performs final, precise selection. Formal complexity analysis shows that HRLO reduces vocabulary projection cost by approximately 64× under default parameters (V=100,000, K=100, r=256). End-to-end speedup depends on the fraction of total compute attributable to vocabulary projection, f_lmhead, and is given by Speedup_e2e = 1 / ((1 − f_lmhead) + f_lmhead/64). For typical transformer deployments where f_lmhead ∈ [0.05, 0.20], this yields end-to-end speedups of 1.1×–1.5×, with larger gains achievable for models with very large vocabularies or highly optimized attention. A directional veto feedback mechanism enables self-correcting inference without additional full model forward passes. HRLO requires no modification of base model weights and is compatible with greedy decoding on any pretrained model exposing final hidden states, subject to the vocabulary-specificity qualification in Section 3.2. Compatibility with sampling-based decoding strategies is discussed in Section 3.5.4.

1. Introduction

Large language models have achieved remarkable capabilities across a wide range of natural language tasks. However, their inference cost remains a critical barrier to deployment at scale. Each autoregressive decoding step requires a full forward pass through the model, culminating in a projection from the model's hidden dimension d to the full vocabulary space V — a matrix

multiplication of shape [d × V] followed by softmax normalization over potentially 100,000 or more tokens.

This output layer constitutes a disproportionately large fraction of per-token inference cost, particularly for models with large vocabularies or when operating on hardware with limited memory bandwidth. Despite this, most inference acceleration research has focused on attention optimization (FlashAttention, KV cache compression) or speculative decoding, leaving the vocabulary projection bottleneck comparatively underexplored as an architectural target.

We propose HRLO, a framework that addresses this gap. HRLO introduces a learned, hierarchical filtering system external to the base model that progressively narrows the token candidate set before M_0's final selection step. Unlike speculative decoding, HRLO does not attempt to predict the exact next token — it aims only to eliminate tokens M_0 would not select. Unlike adaptive softmax, HRLO requires no modification of the base model's output layer. The framework is compatible with greedy decoding on any pretrained model exposing final hidden states.

The remainder of this paper is organized as follows. Section 2 surveys related work. Section 3 presents the HRLO architecture in full. Section 4 provides formal complexity analysis. Section 5 describes planned experiments. Section 6 discusses limitations. Section 7 concludes.

2. Background and Related Work

2.1 The Vocabulary Projection Bottleneck

In standard transformer-based LLMs, the final hidden state $h \in \mathbb{R}^d$ is projected to vocabulary logits via $z = h \cdot W^T$ where $W \in \mathbb{R}^{V \times d}$, followed by softmax normalization. For d=4,096 and V=100,000 this requires 4×10^8 multiply-accumulate operations per token. Yang et al. (2018) formalize this bottleneck theoretically, proving that the standard softmax output layer cannot represent arbitrary next-token distributions when the model's hidden dimension is smaller than the rank of the true contextual probability matrix — the softmax bottleneck. HRLO does not resolve this expressivity limit; it reduces the computational cost of evaluating the distribution by restricting the projection to K candidates, which is orthogonal to the problem Yang et al. identify.

Zhang et al. (2024) confirm this bottleneck empirically at scale, demonstrating that vocabulary projection cost grows significantly with vocabulary size and proposing kernel-level optimizations to mitigate it. Their approach optimizes how the projection executes; HRLO reduces what it operates over. These are complementary.

2.2 Adaptive Softmax

Grave et al. (2017) demonstrated that hierarchically clustering the vocabulary by token frequency can yield significant speedups with minimal accuracy loss. Their adaptive softmax is the closest prior work in spirit to HRLO but requires modification of the model's output layer at training time

and is not post-hoc applicable to pretrained models. HRLO achieves a similar filtering effect externally without touching the base model.

2.3 Speculative Decoding

Speculative decoding (Leviathan et al., 2023; Chen et al., 2023) accelerates inference using a smaller draft model to propose candidate tokens verified in parallel by the base model. Methods such as Medusa (Cai et al., 2024) demonstrated 2×–3× speedups but require a separate draft model or fine-tuning of the base model. HRLO does not predict token sequences — it filters the candidate space for a single position and requires no base model modification.

2.4 Dynamic Vocabulary Pruning

Vincenti et al. (2024) propose dynamically pruning the vocabulary at a fixed intermediate layer inside the model's forward pass, producing a static reduced vocabulary for all subsequent layers within that step. HRLO differs in three respects: it operates entirely outside the base model without accessing intermediate layers, applies a dynamic multi-stage cascade rather than a single fixed pruning point, and employs a veto-driven key correction that adapts the candidate set based on M_0's confidence — making HRLO post-hoc and model-agnostic where Vincenti et al. require internal model access.

2.5 KV Cache and Attention Optimization

KV cache compression (Zhang et al., 2023; Nawrot et al., 2024) and attention approximation methods target the attention mechanism rather than the output projection. These are orthogonal to HRLO and can be combined with it for potentially multiplicative gains.

3. HRLO Architecture

3.1 Overview and Core Geometric Hypothesis

HRLO introduces a hierarchy of N lightweight auxiliary layers $\{M_1, M_2, ..., M_N\}$ trained to approximate M_0's output distribution, combined with a top-level direction key mechanism and a veto feedback loop. At inference, the system runs top-down, progressively narrowing the candidate token set. M_0 is invoked only for final selection over the filtered subset C_1 of size K. Figure 1 illustrates the full inference flow.

The key mechanism rests on a core empirical hypothesis: that cosine similarity in the token embedding matrix of the base model meaningfully clusters contextually plausible next tokens together, such that the correct next token tends to occupy a directional neighborhood of k. We are not aware of published work directly characterizing the geometric structure of transformer output embedding matrices in modern LLMs, and we do not claim this property is established. It is an assumption. The assumption is most likely to degrade for rare tokens where embedding space coverage is sparse and directional structure is weakest — exactly the case where HRLO's filtering

is most sensitive to failure. The rarity bias in Section 3.5.1 partially compensates for this weakness but does not eliminate it.

This hypothesis is directly testable and Section 5.1 is its primary empirical test: the token agreement rate experiment measures precisely whether the correct token falls within the top-K filtered set across diverse generation contexts. High agreement validates the geometric clustering assumption for the target model; low agreement at small K falsifies it. Section 5.1 should therefore be interpreted not only as a quality preservation test but as the primary empirical validation of HRLO's geometric foundation. If the assumption fails for a given model, HRLO is not applicable to that model without architectural revision.

Figure 1: HRLO inference flow — top-down filtering cascade with rarity-biased scoring and veto feedback loop. See hrlo_figure1.svg.

3.2 Model-Agnosticism: Scope and Qualification

HRLO requires no modification to M_0's weights and is compatible with any model exposing its final hidden state $h \in \mathbb{R}^d$ at inference time. However, the auxiliary layers are vocabulary-specific: their output projection $W_2^n \in \mathbb{R}^{V \times r}$ is tied to a specific vocabulary size V. Deploying HRLO on a model with a different tokenizer or vocabulary size requires retraining the auxiliary hierarchy from scratch. The model-agnostic claim therefore applies to inference-time compatibility with a fixed vocabulary, not to transferability across tokenizers.

3.3 Auxiliary Layer Architecture

Each auxiliary layer M_n is a low-rank MLP with GELU activation:

$$M_n(h) = W_2^n \cdot GELU(W_1^n \cdot h)$$

where $W_1^n \in \mathbb{R}^{r \times d}$ projects down to rank r and $W_2^n \in \mathbb{R}^{V \times r}$ projects up to vocabulary size V. Default rank r=256 gives each layer approximately $2 \cdot r \cdot d \approx 2M$ parameters for d=4,096 — roughly 200× smaller than a full lm_head.

GELU activation is chosen over ReLU deliberately. ReLU produces hard zeros for negative activations, creating sparse intermediate representations that risk zeroing out weak signals from rare tokens. GELU's smooth non-zero gradient for negative inputs preserves weak signals and is consistent with the activation functions used in most modern base models. The advantage of GELU over ReLU for rare token preservation is contingent on the same geometric embedding assumption stated in Section 3.1 — specifically, that rare tokens produce systematically weaker activations in the hidden state due to sparse embedding space coverage. If Section 5.1 reveals that the geometric assumption fails for a given model, the activation function choice should be revisited empirically for that deployment, as the GELU rationale does not hold independently of the embedding structure assumption.

The scoring function applies the same $\lambda = 0.1$ at every layer of the hierarchy. This is an implicit architectural assumption that is not theoretically justified. Upper layers performing coarse elimination over large candidate sets may benefit from a higher rarity weight to prevent rare tokens

from being eliminated early when the cosine signal is weakest; lower layers performing fine-grained discrimination over small candidate sets may benefit from a lower rarity weight as the directional signal becomes more precise. A layer-dependent λ_n schedule is the natural extension but is not explored in this preliminary proposal due to the additional hyperparameter complexity it introduces. Uniform λ is retained as the simplest defensible default and is acknowledged as a limitation.

The number of layers N is derived from the per-layer candidate reduction ratio ρ:

$$N = \log(V/K) / \log(1/\rho)$$

For default parameters V=100,000, K=100, ρ=0.1: N=3 layers. The choice ρ=0.1 produces aggressive 10× filtering per layer; the veto mechanism compensates for premature filtering. For gentler progressive refinement, ρ=0.5 yields N=10 layers at greater sequential overhead. Sensitivity analysis across $\rho \in \{0.05, 0.1, 0.3, 0.5\}$ is reported in Section 5.

3.4 Training Procedure

3.4.1 Bottom-Up Distillation

The auxiliary layers are trained sequentially from bottom to top. M_1 distills from M_0, M_2 distills from M_1, and so on. A formal bound on KL error propagation across distillation steps is not straightforward — KL divergence does not satisfy the triangle inequality and standard information-theoretic tools such as the data processing inequality require assumptions that do not cleanly apply here. We treat the additive approximation $\varepsilon_n \leq \varepsilon_{n-1} + \delta_n$ as a working assumption rather than a derived bound, where δ_n is the additional error at layer n. Under this assumption, cumulative error at M_N is at most $N \cdot \delta$ — linear in depth, not exponential. For N=3, this suggests compounding is likely tolerable if per-layer distillation error δ is small, but this is a heuristic argument, not a proof. The actual validation is empirical: Section 5.1 measures token agreement rate per layer — between M_1 and M_0, M_2 and M_0, and M_3 and M_0 separately — to detect whether miscalibration is detectable across layers. Monotonically degrading agreement indicates compounding; stable agreement validates the sequential schedule. Joint training of all auxiliary layers simultaneously would eliminate compounding risk entirely and is deferred to future work.

3.4.2 Loss Function

Each auxiliary layer is trained via forward KL divergence:

$$L(M_n) = KL(\ \mathrm{softmax}(z_0/T)\ \|\|\ \mathrm{softmax}(z_n/T)\ )$$

where T=2 is the distillation temperature (Hinton et al., 2015). Forward KL is chosen deliberately over reverse KL: reverse KL's mode-seeking behavior would cause auxiliary layers to concentrate on peaks of M_0's distribution, systematically underweighting rare tokens. Forward KL penalizes the student for assigning low probability anywhere the teacher assigns high probability, which is the correct priority for a filtering system that must preserve rare token coverage. The KL divergence is computed over the full vocabulary V at training time; savings materialize only at inference.

3.4.3 Training Data and Cost

The default training corpus is C4 (Raffel et al., 2020). For domain-specific deployments, sampling from the target domain corpus is recommended; if unavailable, C4 remains the fallback with the caveat that auxiliary layer calibration may degrade on out-of-distribution inputs. Total training cost is dominated by generating soft labels: approximately N forward passes through M_0 over the training corpus are required. We treat the minimum sufficient corpus size S for adequate calibration as an open empirical question.

3.5 Inference Procedure

3.5.1 Top-Down Filtering with Rarity Bias

At each token generation step:

1. The top layer M_N generates a direction key $k \in \mathbb{R}^d$.
2. Candidate tokens are scored using a convex combination of cosine similarity and rarity bias: $score(t, k) = (1 - \lambda)\cdot sim(E_t, k) + \lambda\cdot rare(t)$, where $sim(E_t, k) = (E_t\cdot k) / (\|E_t\|\cdot\|k\|)$ and $rare(t) = 1 - \log(freq(t) + \varepsilon) / \log(freq_{max} + \varepsilon)$. Rarity scores are precomputed from corpus frequency statistics and stored as a fixed lookup vector; they must be recomputed whenever the training corpus changes. Rarity scores are derived from C4 corpus statistics while the cosine similarity term uses M_0's token embedding geometry; for models trained on substantially different distributions than C4, this introduces a potential misalignment — tokens rare in C4 but common in M_0's training data may receive inflated rarity bias, systematically distorting the scoring function toward the wrong tokens. For such deployments, rarity scores should be recomputed from a corpus representative of M_0's training distribution rather than C4. The scoring function applies the same λ at every layer; the implications of this uniform assumption are discussed in Section 3.3. The top $|C_N| = \rho\cdot V$ tokens by score are retained.
3. Each successive layer M_{n-1} narrows C_n to C_{n-1} with $|C_{n-1}| = \rho\cdot|C_n|$, using the same scoring function.
4. M_0 evaluates the final candidate set C_1 of size K with full precision and selects the output token.
5. If confidence falls below threshold (veto condition), feedback propagates upward and the cascade reruns with a corrected key.

3.5.2 Confidence Metric and Veto Trigger

Confidence is measured by the K-invariant metric:

$$conf(C_1) = (\max(softmax(z)) - 1/K) / (1 - 1/K)$$

This is 0 under uniform uncertainty and 1 under full certainty, normalized to be independent of K. The veto triggers when $conf(C_1) < \tau = 1/K$.

3.5.3 Veto Correction Mechanism

When the veto fires, cluster cohesion of the rejected candidates is assessed:

$$cohesion(C_1) = (2 / K(K-1)) \cdot \Sigma_{i<j} sim(E_{ti}, E_{tj})$$

- cohesion $\geq \delta$: key is updated toward $t_best = \text{argmax}_{t \in C_1} z_t$ using $k_new = k + \alpha_i \cdot (E_{t_best} - k) / \|E_{t_best} - k\|$, where $\alpha_i = \alpha_0 \cdot \gamma^i$, $\alpha_0 = 0.1 \cdot \|k\|$, $\gamma = 0.5$ (Armijo, 1966).
- cohesion $< \delta$: candidates are scattered; K is expanded to min(2K, K_max). If K > K_max, full M_0 evaluation over V is triggered — correctness guaranteed, speedup forfeited for that token.

The veto loop is not guaranteed to converge in non-convex embedding regions. The hard fallback at K > K_max bounds worst-case cost. Under worst-case δ miscalibration, the system falls back to full M_0 evaluation, yielding 1.0× speedup — no gain, no degradation below baseline.

3.5.4 Compatibility with Decoding Strategies

The inference procedure assumes greedy decoding. Compatibility with other strategies is as follows.

Temperature sampling over C_1 is approximately compatible with HRLO with one important caveat: if the correct token was filtered out of C_1, temperature sampling cannot recover it, and the error is silent. The confidence metric conf(C_1) measures peakedness of the distribution over whatever is in C_1, not whether the globally correct token is present. A veto can fire when C_1 is internally incoherent but cannot fire when C_1 is coherent yet missing the correct token. For temperature sampling deployments, larger K is recommended to reduce silent exclusion errors.

Top-p (nucleus) sampling is incompatible with HRLO in its current form. Nucleus sampling selects from the smallest token set whose cumulative probability exceeds p under the full vocabulary distribution. If HRLO's C_1 excludes tokens that would have appeared in the nucleus, the effective nucleus is distorted with no veto fired. Correct integration would require computing the nucleus boundary before filtering, partially defeating the purpose. This is deferred to future work.

Beam search requires evaluating multiple hypotheses simultaneously. HRLO generates a single C_1 per decoding step. Whether to share C_1 across all beams or run separate filtering passes per beam is an open design question: shared C_1 reduces cost but may disadvantage semantically divergent beams; per-beam filtering preserves quality but multiplies auxiliary layer cost by beam width. Beam search compatibility is out of scope for this preliminary proposal.

In summary: HRLO as specified is compatible with greedy decoding and approximately compatible with temperature sampling at larger K. Top-p sampling and beam search require architectural extensions not addressed here.

3.6 Hyperparameter Summary

HRLO introduces eight tunable quantities. Two (τ, α_0) are derived from existing variables. The remaining six are fixed constants set by principled reasoning rather than grid search.

Param	Default	Type	Role	Justification
$\tau = 1/K$	Scales with K	Derived	Veto threshold	Uniform-distribution baseline; directly interpretable
$\alpha_0 = 0.1 \cdot \|k\|$	Scales with $\|k\|$	Derived	Initial step size	Proportional to key magnitude for geometric consistency
$\gamma = 0.5$	Fixed	Constant	Step decay rate	Canonical backtracking line search (Armijo, 1966)
$\delta = 0.3$	Fixed	Constant	Cohesion threshold	Worst-case miscalibration yields 1.0×; no degradation below baseline
K_max = 1000	Fixed	Constant	Fallback trigger	10× largest tested K; any value >> 500 serves equally
$\rho = 0.1$	Fixed	Constant	Reduction ratio	Aggressive filtering; veto compensates; sensitivity in Section 5
r = 256	Fixed	Constant	Layer rank	200× smaller than lm_head; sufficient expressivity
$\lambda = 0.1$	Fixed	Constant	Rarity weight	Residual correction only; directional key handles primary signal; uniform λ acknowledged as limitation in Section 3.3

3.7 Memory Overhead

Each auxiliary layer requires r·(d+V)·4 bytes in float32:

```
Memory_layer = 256 · (4096 + 100000) · 4 ≈ 106 MB
Memory_total (N=3) ≈ 318 MB
```

A 7B base model in float16 occupies approximately 14GB; auxiliary layers add approximately 2% overhead. Overhead scales with V rather than model size. For V=500,000, memory per layer approximately doubles. Deployments combining large vocabularies with memory-constrained hardware should reduce r or quantize auxiliary weights independently of M_0.

4. Computational Complexity Analysis

4.1 Full Per-Token Cost

At layer n, the candidate set size is $|C_n| = K\cdot(1/\rho)^{(N-n)}$. Full HRLO cost per token (single pass, no veto):

```
Cost_HRLO = Σ_{n=1}^{N} O(d·r + r·|C_n|) + O(d·K)
```

The geometric sum $\Sigma|C_n| = (V-K)\cdot\rho/(1-\rho) \approx V\cdot\rho/(1-\rho)$ under the condition $K \ll V$ (approximation error K/V = 0.1% for default parameters; for K/V > 0.01 use the exact expression). Simplified:

$$\text{Cost_HRLO} = O(N \cdot d \cdot r + r \cdot V \cdot \rho/(1-\rho) + d \cdot K)$$

4.2 Speedup Derivation

Standard decoding cost: $O(d \cdot V)$. Formal speedup on vocabulary projection:

$$\text{Speedup_proj} = d \cdot V \;/\; (N \cdot d \cdot r + r \cdot V \cdot \rho/(1-\rho) + d \cdot K)$$

Substituting defaults ($d=4096$, $V=100000$, $K=100$, $r=256$, $\rho=0.1$, $N=3$): numerator = 409,600,000; denominator = 3,145,728 + 2,844,444 + 409,600 = 6,399,772.

$$\text{Speedup_proj} \approx 64\times \text{ on vocabulary projection}$$

End-to-end speedup as a function of f_lmhead (fraction of total per-token compute on vocabulary projection):

$$\text{Speedup_e2e}(f_lmhead) = 1 \;/\; ((1 - f_lmhead) + f_lmhead/64)$$

Note: the theoretical analysis is parameterized around V=100,000. The primary experimental model LLaMA 3 8B uses V=128,000, which yields marginally higher Speedup_proj. Readers may substitute V=128,000 directly into the formula for LLaMA 3-specific projections.

5. Planned Experiments

All experiments use LLaMA 3 8B (V=128,000) as the primary model and GPT-2 (V=50,257) as an accessible secondary baseline. GPT-2 f_lmhead measurements are scaled by V_GPT2/V_theoretical = 50,257/100,000 ≈ 0.50 before application to the speedup formula, as lm_head cost scales linearly with V. Hardware: single NVIDIA A100 80GB GPU; CPU results reported separately.

5.1 Token Agreement Rate

The primary empirical question: if HRLO restricts M_0's softmax to top-K tokens, how often is the correct token preserved? This is simultaneously a quality preservation test and the primary empirical validation of HRLO's geometric embedding assumption (Section 3.1). We simulate HRLO by restricting softmax to top-K at inference and measure:

- Token agreement rate: fraction of steps where top-K restricted output matches full-vocabulary greedy output.
- Per-layer agreement: between M_1 and M_0, M_2 and M_0, and M_3 and M_0 separately, to detect error compounding across distillation layers.
- Mean rank of correct token across generation steps.
- Coverage across $K \in \{50, 100, 200, 500\}$.

5.2 Sensitivity Analysis

- $\rho \in \{0.05, 0.1, 0.3, 0.5\}$: varying hierarchy depth and per-layer aggressiveness, measuring token agreement rate and theoretical speedup.

- K ∈ {50, 100, 200, 500}: varying candidate set size. K should be tuned per deployment — code generation and multilingual models exhibit higher rare token density than conversational models and will likely require larger K.

5.3 End-to-End Speedup as Function of f_lmhead

f_lmhead will be profiled directly under: batch size ∈ {1, 8, 32}; sequence length ∈ {128, 512, 1024}; attention implementation: standard PyTorch and FlashAttention-2. f_lmhead is measured as the fraction of total per-token wall-clock time attributed to the lm_head linear projection, isolated using PyTorch profiler with CUDA event timing. We are not aware of published profiling studies reporting this fraction directly for modern LLM architectures; empirical measurements will replace the bracketed range {0.05, 0.40} with model- and condition-specific values.

5.4 Evaluation Benchmarks

Quality preservation will be evaluated on three benchmarks: MT-Bench (conversational quality), GSM8K (mathematical reasoning), and HumanEval (code generation). HumanEval is included specifically because the paper identifies code generation as a highest-risk deployment context for rare token failure — code contains identifiers, keywords, and syntax tokens with highly non-uniform frequency distributions that stress the rarity bias and filtering assumptions more severely than conversational text. A multilingual benchmark (FLORES-200 or equivalent) is noted as a further extension; it is deferred due to resource constraints but acknowledged as a gap given that multilingual text is identified as the second highest-risk deployment context.

6. Limitations and Open Questions

- Geometric embedding assumption: the entire key mechanism assumes cosine similarity in M_0's token embedding matrix clusters contextually plausible tokens. This is unproven and Section 5.1 is its primary test. If it fails for a given model, HRLO is inapplicable without revision.
- Corpus-model alignment: rarity scores are computed from C4 while the cosine similarity term uses M_0's embedding geometry. For models trained on distributions substantially different from C4, these two components may be misaligned. Rarity scores should be recomputed from a corpus representative of M_0's training distribution.
- Uniform λ across layers: the same rarity weight is applied at every layer despite the filtering task changing from coarse elimination to fine-grained discrimination. A layer-dependent λ_n schedule is the natural extension, deferred to future work.
- Vocabulary-specific auxiliary layers: redeployment on a model with a different tokenizer requires retraining the full auxiliary hierarchy.
- Error compounding in sequential distillation: the $N \cdot \delta$ error bound is a heuristic assumption; per-layer agreement measurement in Section 5.1 is the empirical test.
- Greedy decoding only: temperature sampling has a silent exclusion failure mode; top-p sampling and beam search require architectural extensions not addressed here.

- Veto loop non-convergence: no guarantee in non-convex embedding regions. Hard fallback at K > K_max bounds worst-case at 1.0× speedup.
- f_lmhead model-specificity: end-to-end speedup requires model-specific profiling. The formula is provided; universal values are not.
- GELU advantage contingent on geometric assumption: if the embedding structure assumption fails, the activation function choice requires revisiting.
- Multilingual evaluation deferred: acknowledged gap given multilingual text is identified as high-risk.

7. Conclusion

We have presented HRLO, a model-agnostic inference acceleration framework targeting the vocabulary projection bottleneck in LLM decoding. Key contributions: (1) a bottom-up trained, top-down executed filtering hierarchy using low-rank GELU MLPs with rarity-biased candidate scoring; (2) a K-invariant confidence metric and veto trigger; (3) a cohesion-gated key correction mechanism pulling toward positive signal with exponential decay step size; (4) a hard fallback guaranteeing correctness with 1.0× minimum speedup under any parameter miscalibration; (5) compatibility with greedy decoding on any model exposing final hidden states, with decoding strategy limitations explicitly scoped. Formal analysis yields 64× vocabulary projection reduction and 1.1×–1.5× end-to-end speedup for typical deployments. The core geometric embedding assumption is explicitly stated as unproven; Section 5.1 constitutes its primary empirical test, with HumanEval and GSM8K covering the highest-risk deployment contexts identified.

References

Armijo, L. (1966). Minimization of functions having Lipschitz continuous first partial derivatives. Pacific Journal of Mathematics, 16(1), 1–3.

Cai, T., et al. (2024). Medusa: Simple framework for accelerating LLM generation with multiple decoding heads. arXiv:2401.10774.

Chen, C., et al. (2023). Accelerating large language model decoding with speculative sampling. arXiv:2302.01318.

Dao, T. (2024). FlashAttention-2: Faster attention with better parallelism and work partitioning. ICLR 2024.

Grave, E., Joulin, A., Cisse, M., Grangier, D., & Jégou, H. (2017). Efficient softmax approximation for GPUs. ICML 2017.

Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. arXiv:1503.02531.

Leviathan, Y., Kalman, M., & Matias, Y. (2023). Fast inference from transformers via speculative decoding. ICML 2023.

Nawrot, P., et al. (2024). Dynamic memory compression: Retrofitting LLMs for accelerated inference. ICML 2024.

Raffel, C., et al. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. JMLR, 21(140), 1–67.

Vincenti, J., Abdel Sadek, K., Velja, J., Nulli, M., & Jazbec, M. (2024). Dynamic vocabulary pruning in early-exit LLMs. arXiv:2410.18952.

Yang, Z., et al. (2018). Breaking the softmax bottleneck: A high-rank RNN language model. ICLR 2018.

Zhang, J., et al. (2024). From projection to prediction: Beyond logits for scalable language models. arXiv:2511.17599.

Zhang, Z., et al. (2023). H2O: Heavy-hitter oracle for efficient generative inference of large language models. NeurIPS 2023.